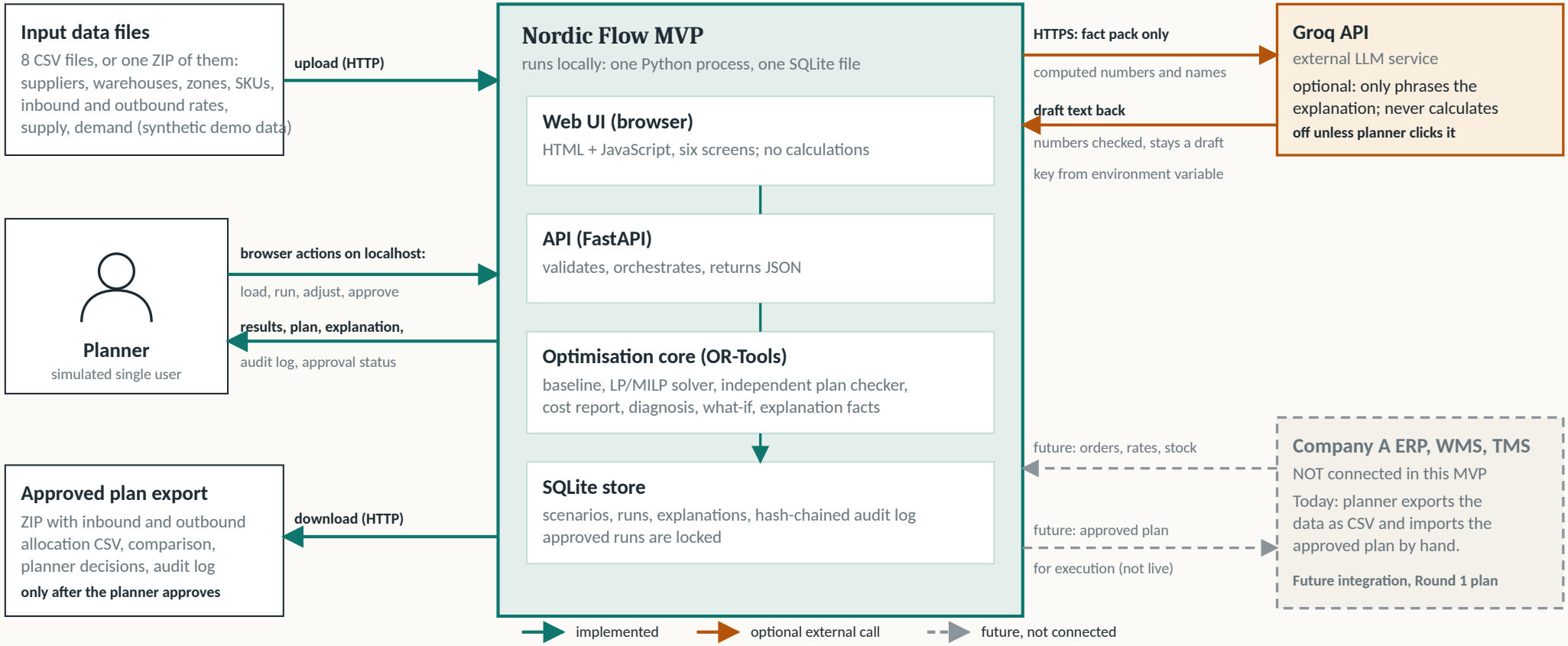
System context: Nordic Flow MVP and everything around it

What is inside the system boundary, what is outside, and every connection between them. Solid lines are implemented and demonstrated; the dashed box is future integration and is not connected.



INSIDE THE BOUNDARY

- Web UI, API, optimisation core, SQLite store
- All calculation, verification and approval rules
- The audit log and run history

OUTSIDE THE BOUNDARY

- The planner and their browser
- Input files and the exported plan
- Groq (optional), Company A ERP, WMS and TMS (future)

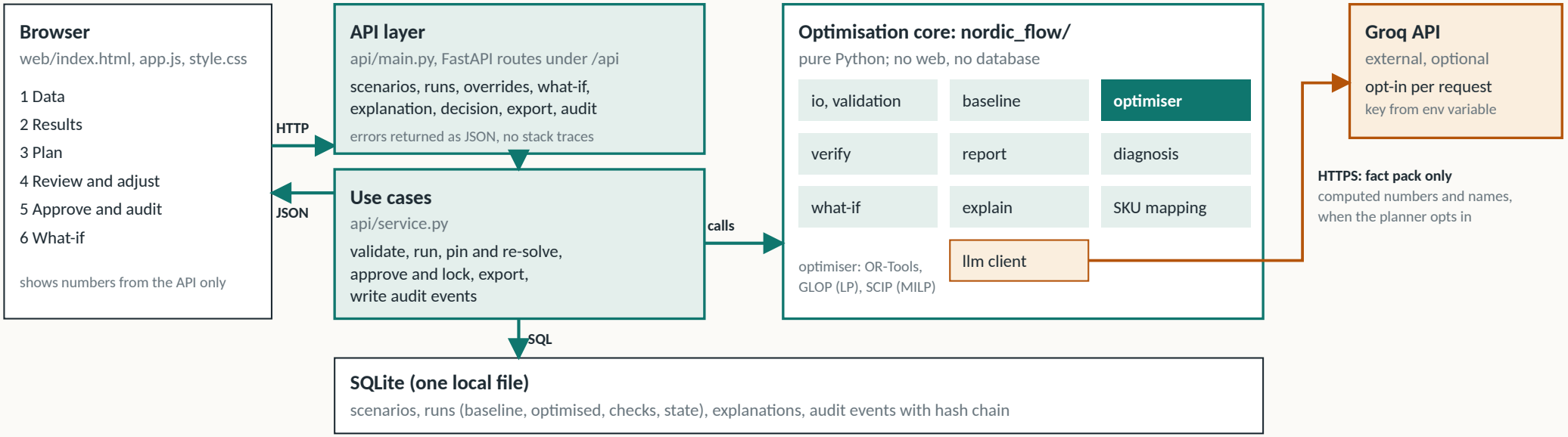
FACTS ABOUT THIS MVP

- No live link to any Company A system: CSV in, CSV out
- Nordic Flow proposes; nothing is executed or written back
- Data is synthetic; no login (one simulated planner)

Reflects the running application as of this submission.

Inside the system: components, data flow and trust gates

How the parts connect when a planner runs and approves a plan. The optimisation core has no web or database code; the API layer has no calculation logic.



One run, step by step

1 Validate. Files parsed, checked by file and row. Errors stop here.

2 Baseline. Preferred-warehouse rule builds the comparison plan.

3 Optimise. LP or MILP finds the lowest-cost plan; status recorded.

4 Verify. Separate checker re-tests both plans against every rule.

5 Compare. Cost by type, load per warehouse; saving only if baseline valid.

6 Store. Run and audit event saved; planner reviews and decides.

TRUST GATE	WHAT IT DOES	WHERE IT LIVES
Independent verifier	Re-checks supply, demand, capacity, lanes and balance; a plan that leaves demand unmet is invalid	nordic_flow/verify.py
No saving on an unusable baseline	If the baseline cannot serve all demand, saving is null and shown as "n/a"	api/service.py, nordic_flow/report.py
AI cannot calculate	Facts come from code; LLM only phrases them, text with unknown numbers is discarded, output stays a draft until approved	nordic_flow/explain.py
Approval lock and export gate	An approved or rejected run cannot change; export works only for approved runs	api/service.py
Tamper-evident audit	Each event stores a hash of the previous one, so edits are detectable (not tamper-proof)	api/audit.py

Only components present in the running MVP are drawn as connected.